

گزارش پیشرفت پروژه: توسعه‌ی الگوریتم SCL-Star به ESCL-Star و تعمیم تولیدکننده‌ی آزمون‌های خودکار

آرین باستانی

چکیده

این گزارش، توسعه‌ی الگوریتم یادگیری ترکیبی SCL-Star به ESCL-Star را مستند می‌کند: تعمیمی که به کنش‌های هم‌گام‌ساز اجازه می‌دهد به‌جای یک خروجی ایستای ثابت، خروجی‌های متفاوت اما سازگار در نقاط هم‌گامی تولید کنند. علاوه بر شرح صوری و پیاده‌سازی الگوریتم، تعمیم متناظر تولیدکننده‌ی آزمون‌های خودکار نیز با جزئیات کامل ارائه می‌شود. در پایان، نتایج آزمایش‌ها روی آزمون‌های تولیدی برای چهار توپولوژی و نیز یک بنچمارک واقعی تازه (bankCard) گزارش می‌شود که نشان می‌دهد مزیت یادگیری ترکیبی نسبت به یادگیری یکپارچه، پس از این تعمیم همچنان حفظ می‌شود.

فهرست مطالب

۱	مقدمه	۳
۲	مروری بر محدودیت الگوریتم پیشین	۳
۳	الگوریتم ESCL-Star	۳
۱.۳	ایده‌ی اصلی	۳
۲.۳	تعاریف پایه	۵
۳.۳	پیاده‌سازی نهایی: merge، makeSet و ProductMealy	۶
۴	تعمیم تولیدکننده‌ی آزمون‌های خودکار	۷
۱.۴	صورت مسئله	۷
۲.۴	استخراج کوچک‌ترین دوره‌ی تناوب	۷
۳.۴	ساخت اسکلت سازگار با الگو	۸
۴.۴	الحاق حالت‌های اضافی بدون نقض الگو	۹
۵.۴	خنثی‌سازی اثر کنش‌های خصوصی بر الگوی هم‌گامی	۱۰
۶.۴	تصادفی‌سازی نوع مؤلفه‌ها در آزمون‌های نقطه‌به‌نقطه	۱۰
۵	آزمایش‌ها و نتایج بر روی آزمون‌های تولیدشده	۱۰
۶	بنچمارک واقعی: bankCard	۱۱
۷	جمع‌بندی	۱۲

فهرست تصاویر

۱	الگوریتم پردازش ضدمثال در SCL-Star: به محض مشاهده‌ی یک خروجی متفاوت برای کنش هم‌گام‌ساز u ، کنش بدون بررسی بیشتر از sync حذف می‌شود.	۴
۲	پیش‌نویس اولیه‌ی الگوریتم پردازش ضدمثال در ESCL-Star: به جای حذف کنش، مجموعه‌های الفبایی مرتبط با آن یاد گرفته و صحت هم‌گامی آن‌ها با LearnSyncSets بررسی می‌شود.	۵
۳	مقایسه‌ی تعداد نمادهای پرسش عضویت (چپ) و تعداد بازنشانی‌ها (راست) برای LSTAR در برابر ESCL-Star روی مجموعه‌ی ادغام‌شده‌ی آزمون‌های تولیدشده.	۱۱
۴	نمودارهای مقایسه‌ی تعداد نمادها، تعداد بازنشانی‌ها، و نمای ادغام‌شده برای ESCL-Star روی بنچمارک واقعی bankCard.	۱۲

فهرست جداول

۱	میانگین معیارهای یادگیری روی آزمون‌های تولیدشده: مقایسه‌ی آزمون‌های پیشین (خروجی ایستا) با آزمون‌های تازه (خروجی پویا)	۱۱
۲	میانگین معیارهای یادگیری روی بنچمارک واقعی: بنچمارک پیشین در برابر bankCard	۱۲

فهرست الگوریتم‌ها

۱.۳	پردازش ضدمثال در ESCL-Star (خلاصه‌ی پیاده‌سازی ProcessCE)	۶
۱.۴	getMinimalPattern: کاهش یک الگو به کوچک‌ترین دوره‌ی تناوب	۸
۲.۴	ساخت اسکلت گراف بر مبنای الگوی خروجی هم‌گام‌ساز	۸
۳.۴	الحاق حالت‌های اضافی به اسکلت سازگار با الگو	۹

۱ مقدمه

این گزارش، توسعه‌هایی را مستند می‌کند که از تعهد 4844f2f تا انتهای تاریخچه‌ی فعلی مخزن ESCL-Star پیاده‌سازی شده‌اند. محور اصلی کار، رفع یک محدودیت بنیادین در الگوریتم یادگیری ترکیبی SCL-Star^۲ است: در نگارش پیشین، هر کنش هم‌گام‌ساز^۳ می‌بایست در تمام مؤلفه‌ها و در تمام حالت‌هایی که رخ می‌دهد، دقیقاً یک خروجی ایستا و یکسان تولید کند. الگوریتم توسعه‌یافته که در این گزارش ESCL-Star^۴ نامیده می‌شود، این محدودیت را برمی‌دارد و اجازه می‌دهد یک کنش هم‌گام‌ساز، در نقاط مختلف اجرای سامانه، خروجی‌های متفاوتی داشته باشد؛ به شرط آنکه در هر لحظه‌ی هم‌گامی، تمام مؤلفه‌های شرکت‌کننده در آن هم‌گامی، خروجی یکسانی برای آن کنش تولید کنند. این شرط، برای حفظ قطعیت^۵ ترکیب موازی سامانه‌ی هدف ضروری است.

پس از تشریح الگوریتم ESCL-Star، گزارش به بخش دوم و پراهمیت‌تر کار می‌پردازد: تعمیم تولیدکننده‌ی آزمون‌های خودکار^۶ به گونه‌ای که بتواند مؤلفه‌های تصادفی‌ای تولید کند که کنش‌های هم‌گام‌ساز آن‌ها می‌توانند خروجی‌های متفاوت داشته باشند، اما همواره در نقاط هم‌گامی با یکدیگر سازگارند. طراحی این تولیدکننده از دشوارترین بخش‌های این کار بوده و در بخش ۴ با جزئیات کامل تشریح می‌شود. در پایان، نتایج آزمایش‌های اجراشده روی آزمون‌های تولیدی و همچنین یک بنچمارک واقعی جدید (bankCard) گزارش می‌شود.

۲ مروری بر محدودیت الگوریتم پیشین

در SCL-Star، سامانه‌ی هدف $M = \parallel_{i=1}^n M_i$ متشکل از n مؤلفه‌ی موازی هم‌گام است. الگوریتم به صورت برخط^۷ یک پوشش $I^F = \{I_1, \dots, I_n\}$ از الفبای کل سامانه را می‌سازد؛ اگر دو مجموعه‌ی I_i و I_j اشتراک غیرتهی داشته باشند، مؤلفه‌های متناظر روی کنش‌های مشترک هم‌گام می‌شوند. فرض بنیادین نگارش پیشین آن بود که هر کنش هم‌گام‌ساز، خروجی واحدی دارد که در تمام مؤلفه‌ها و تمام حالت‌ها ثابت است (مثلاً کنش «a» همیشه خروجی «1» تولید می‌کند). این فرض، تشخیص نادرستی کنش‌های هم‌گام‌ساز را ساده می‌کرد: کافی بود در طول یادگیری، خروجی هر کنش هم‌گام‌ساز رصد و در صورت مشاهده‌ی هر مقدار متفاوت، فوراً آن کنش از مجموعه‌ی sync حذف شود. این فرض در عمل بسیاری از سامانه‌های واقعی را پوشش نمی‌دهد؛ برای نمونه، در پروتکل‌های کارت بانکی (بخش ۶)، یک کنش می‌تواند بسته به وضعیت داخلی مؤلفه (مثلاً وضعیت احراز هویت) پاسخ‌های متفاوتی صادر کند، در حالی که همچنان کنشی هم‌گام‌ساز میان چند مؤلفه محسوب می‌شود. هدف این پروژه، تعمیم الگوریتم برای پوشش چنین رفتاری بدون از دست دادن قطعیت مدل ترکیبی بود.

۳ الگوریتم ESCL-Star

۱.۳ ایده‌ی اصلی

در ESCL-Star کنش هم‌گام‌ساز u مجاز است در نقاط مختلف اجرا، خروجی‌های متفاوتی تولید کند؛ تنها الزام آن است که هرگاه دو یا چند مؤلفه هم‌زمان روی u هم‌گام می‌شوند، خروجی تولیدی هر یک از آن‌ها برای u در آن نقطه‌ی

¹4844f2f754af13a63b5aacaad668fd637572c339

²Synchronous Compositional Learning

³synchronizing action

⁴Extended SCL-Star

⁵determinism

⁶Generated Tests

⁷on-the-fly

هم‌گامی، دقیقاً برابر باشد. این شرط، عدم قطعیت خروجی در ترکیب موازی دو ماشین Mealy قطعی را از بین می‌برد و ضامن صحت ترکیب است.

از منظر پیاده‌سازی، تفاوت اصلی با نگارش پیشین در نحوه‌ی پردازش ضدمثال^۸ است. در نگارش پیشین، به محض مشاهده‌ی دو خروجی متفاوت برای یک کنش هم‌گام‌ساز، آن کنش به‌سادگی از sync حذف می‌شد (شکل ۱). این کار در حالتی که کنش واقعاً هم‌گام‌ساز است اما خروجی پویا دارد، اشتباه است؛ زیرا کنش را به غلط ناهم‌گام تشخیص می‌دهد. در ESCL-Star (شکل ۲) برای پیش‌نویس اولیه، و پیاده‌سازی نهایی در بخش (۳.۳)، به‌جای حذف کنش از sync، مجموعه‌های الفبایی^۹ حاوی آن کنش را می‌یابیم و آن‌ها را با هم یاد می‌گیریم؛ در صورتی که خروجی کنش هنگام هم‌گامی در مدل یادگرفته‌شده‌ی ترکیبی هم‌چنان ناسازگار باشد، آنگاه واقعاً کنش هم‌گام‌ساز نیست و مجموعه‌های الفبایی متناظر با آن باید ادغام شوند تا در یک مؤلفه‌ی واحد یاد گرفته شوند.

Algorithm 2 Process Counter-Example

Input: CE
Effect: Updating the global variables I^F , $sync$, and $OutSync$

```

1: procedure ProcessCE
2:    $OutCE \leftarrow Output(CE, M)$ 
3:   for each  $u \in sync$  do
4:     if  $\exists i \cdot CE[i] = u \wedge \exists \{u \mapsto x\} \in OutSync \cdot OutCE[i] \neq x$  then
5:        $sync := sync \setminus \{u\}$ 
6:        $OutSync := OutSync \setminus \{u \mapsto x\}$ 
7:        $I^s := \{I_j \mid I_j \in I^F \wedge u \in I_j\}$ 
8:        $I^F \leftarrow Merge(I^F, I^s)$ 
9:    $I^D \leftarrow DependentSets(CE, I^F, sync)$ 
10:  for each  $u_r \in sync$  s.t.  $\exists i \cdot CE[i] = u_r$  do    $I^D \leftarrow I^D \cup \{\{u_r\}\}$ 
11:   $I^F \leftarrow Merge(I^F, I^D)$ 
12: end procedure

```

شکل ۱: الگوریتم پردازش ضدمثال در SCL-Star: به‌محض مشاهده‌ی یک خروجی متفاوت برای کنش هم‌گام‌ساز u ، کنش بدون بررسی بیشتر از sync حذف می‌شود.

^۸counter-example

^۹input sets

Algorithm 2 Process Counter-Example

Input: CE

Effect: Updating the global variables I^F , $sync$

```
1: procedure ProcessCE
2:    $I^D \leftarrow \emptyset$ 
3:    $I_{CE} \leftarrow \text{MakeSet}(CE)$ 
4:   if  $\exists I_i \in I^F \cdot I_i \subseteq I_{CE}$  then  $I^F \leftarrow I^F \setminus I_i$ 
5:    $I^F \leftarrow I^F \cup I_{CE}$ 
6:   for each  $I_j \in I^F \cdot I_{CE} \cap I_j \neq \emptyset \wedge I_j \neq I_{CE}$  do  $I^D \leftarrow I^D \cup I_j$ 
7:   if  $I^D \neq \emptyset$  then  $I^D \leftarrow \text{LearnSyncSets}(I^D \cup I_{CE})$ 
8:   if  $I^D \neq \emptyset$  then  $I^F \leftarrow \text{Merge}(I^F, I^D)$ 
9: end procedure
```

Algorithm 3 Learn Synchronizing Sets

Result: $I^{D'}$ // Set of updated independent covers

Input: I^D // Set of independent covers

```
1: function LearnSyncSets
2:    $I^M \leftarrow \emptyset, I^{D'} \leftarrow \emptyset$ 
3:    $\mathcal{H} \leftarrow \text{LearnSyncInParts}(M, I^D)$ 
4:    $wrongsync \leftarrow \text{CheckOutputs}(\mathcal{H})$ 
5:   if  $wrongsync \neq \emptyset$  then
6:     for each  $u \in wrongsync$  do
7:        $I^M \leftarrow \text{DependentSets}(I^D, wrongaction)$ 
8:        $I^{D'} \leftarrow I^{D'} \cup I^M$ 
9:   return  $I^{D'}$ 
10: end function
```

شکل ۲: پیش نویس اولیه‌ی الگوریتم پردازش ضدمثال در ESCL-Star: به جای حذف کنش، مجموعه‌های الفبایی مرتبط با آن یاد گرفته و صحت هم‌گامی آن‌ها با LearnSyncSets بررسی می‌شود.

۲.۳ تعاریف پایه

- **SymbolsSet**: مجموعه‌ی تمام کنش‌های متمایز حاضر در یک ضدمثال CE را برمی‌گرداند.
- **DependentSets**: از میان پوشش I^F ، مجموعه‌هایی را برمی‌گرداند که حداقل یک کنش مشترک با یک مجموعه‌ی داده‌شده دارند.
- **Merge**: دو پوشش را می‌گیرد و مجموعه‌های مشترکشان را با اجتماع آن‌ها جایگزین می‌کند.
- **LearnSyncInParts**: هر مجموعه‌ی الفبایی $I_i \in I^F$ را با L^* به‌طور مستقل یاد می‌گیرد و ترکیب موازی هم‌گام آن‌ها را برمی‌گرداند.

۳.۳ پیاده‌سازی نهایی: merge، makeSet و ProductMealy

پیاده‌سازی نهایی در فایل src/main/Sc1Star.java این ایده را در سه گام به کار می‌بندد که در الگوریتم ۱.۳ خلاصه شده است:

۱. تابع makeSet مجموعه‌ی I_{CE} را از تمام کنش‌های متمایز حاضر در ضدمثال کمینه^{۱۰} می‌سازد و آن را جایگزین هر مجموعه‌ی موجود در I^F می‌کند که زیرمجموعه‌ی آن است.
۲. مجموعه‌های I^F که با I_{CE} اشتراک دارند اما با آن برابر نیستند، به عنوان I^D (مجموعه‌های وابسته‌ی مشکوک) جدا می‌شوند.
۳. اگر I^D ناتهی باشد، learnSynSets فراخوانی می‌شود: هر مجموعه در $I^D \cup \{I_{CE}\}$ به طور مستقل با L^* یاد گرفته و توسط ProductMealy.mergeFSMs در قالب یک ترکیب هم‌گام واحد ساخته می‌شود. در حین ساخت این ترکیب، هرگاه یک کنش مشترک a در دو مدل جزئی، برای یک حالت مشترک ترکیب شده، دو خروجی متفاوت تولید کند، آن کنش به فهرست wrongSyncs افزوده می‌شود (این بررسی دقیقاً معادل تابع CheckOutputs در تعریف صوری الگوریتم است).
۴. برای هر کنش در wrongSyncs، مجموعه‌های الفبایی حاوی آن (از طریق DependentSets) گرد آورده و با merge در I^F ادغام می‌شوند تا از این پس با هم و در قالب یک مؤلفه‌ی یادگیری واحد آموخته شوند.

الگوریتم ۱.۳ پردازش ضدمثال در ESCL-Star (خلاصه‌ی پیاده‌سازی ProcessCE)

- ورودی: ضدمثال کمینه CE ، پوشش فعلی I^F
 خروجی: پوشش به‌روزشده‌ی I^F
- ۱: $I_{CE} \leftarrow \text{makeSet}(CE)$
 - ۲: اگر $\exists I_i \in I^F$ به طوری که $I_i \subseteq I_{CE}$ آنگاه
 - ۳: $I^F \leftarrow I^F \setminus \{I_i\}$
 - ۴: پایان شرط اگر
 - ۵: $I^F \leftarrow I^F \cup \{I_{CE}\}$
 - ۶: $I^D \leftarrow \{I_j \in I^F \mid I_j \neq I_{CE} \wedge I_j \cap I_{CE} \neq \emptyset\}$
 - ۷: اگر $I^D \neq \emptyset$ آنگاه
 - ۸: $\mathcal{H} \leftarrow \text{LearnSyncInParts}(M, I^D \cup \{I_{CE}\})$ {یادگیری جزئی هر مجموعه با L^* }
 - ۹: $\text{wrongSyncs} \leftarrow$ کنش‌های هم‌گامی که در ساخت ProductMealy خروجی ناسازگار داشته‌اند
 - ۱۰: $I^{D'} \leftarrow \bigcup_{u \in \text{wrongSyncs}} \text{DependentSets}(I^D \cup \{I_{CE}\}, u)$
 - ۱۱: برای هر $I_i \in I^{D'}$ انجام بده
 - ۱۲: $I^F \leftarrow \text{Merge}(I^F, I_i)$
 - ۱۳: پایان حلقه‌ی برای
 - ۱۴: پایان شرط اگر
 - ۱۵: بازگردان I^F
-

نکته‌ی کلیدی آن است که ادغام دیگر بر پایه‌ی «هر خروجی متفاوت برابر است با ناهم‌گام‌سازی» صورت نمی‌گیرد؛ بلکه فقط زمانی رخ می‌دهد که خروجی ناسازگار، در لحظه‌ی واقعی هم‌گامی دو مؤلفه‌ی مستقلاً یادگرفته‌شده ظاهر شود. این تغییر، دقیقاً همان چیزی است که امکان یادگیری صحیح سامانه‌هایی با کنش‌های هم‌گام‌ساز چندخروجی را فراهم می‌کند.

¹⁰minimal counter-example

۴ تعمیم تولیدکنندهی آزمون‌های خودکار

پس از تعمیم الگوریتم یادگیری، لازم بود مجموعه‌ای از آزمون‌های خودکار ساخته شود که واقعاً رفتار موردنظر (کنش هم‌گام‌ساز با خروجی پویا اما سازگار در نقاط هم‌گامی) را در خود داشته باشند؛ در غیر این صورت امکان سنجش عملکرد ESCL-Star در برابر این حالت جدید وجود نداشت. طراحی این تولیدکننده، به‌مراتب پیچیده‌تر از توسعه‌ی الگوریتم یادگیری بود و در این بخش با جزئیات کامل شرح داده می‌شود.

۱.۴ صورت مسئله

هر مؤلفه‌ی هم‌گام‌ساز باید به‌طور کاملاً مستقل و تصادفی تولید شود (توپولوژی، تعداد حالت‌ها، کنش‌های خصوصی^{۱۱} دلخواه)، اما رد خروجی^{۱۲} کنش هم‌گام‌ساز مشترک، از حالت آغازین به بعد، باید در همه‌ی مؤلفه‌های شرکت‌کننده در آن هم‌گامی، دقیقاً یکسان باشد. از آنجا که هر ماشین حالت متناهی قطعی، در نهایت به یک چرخه‌ی دوره‌ای می‌رسد، رد خروجی یک کنش را می‌توان با یک الگو^{۱۳} به‌صورت زوج (پیشوند، دوره‌ی تناوب) نمایش داد:

$$\text{pattern} = (\text{BEFORE_LOOP}, \text{LOOP})$$

که در آن BEFORE_LOOP دنباله‌ای است که فقط یک‌بار پیش از ورود به چرخه طی می‌شود و LOOP کوچک‌ترین بلوک تکرارشونده‌ی چرخه است. چالش اصلی آن است که: نخستین مؤلفه‌ی یک گروه هم‌گام‌ساز، به‌طور کاملاً آزاد تولید و الگوی خروجی آن استخراج می‌شود (تابع generateSynchOutPattern)؛ سپس این الگو به تولیدکننده‌ی هر مؤلفه‌ی بعدی داده می‌شود و آن مؤلفه باید ماشینی کاملاً متفاوت (با توپولوژی، تعداد حالت و کنش‌های خصوصی متفاوت) بسازد که وقتی از حالت آغازینش روی کنش هم‌گام‌ساز طی شود، دقیقاً همان الگو بازتولید شود.

۲.۴ استخراج کوچک‌ترین دوره‌ی تناوب

پیش از تولید، لازم است الگوی یک رشته‌ی خروجی به فرم مینیمال (کوچک‌ترین دوره‌ی تکرارشونده) کاهش یابد؛ در غیر این صورت الگوهایی که صرفاً چندبرابر یک دوره‌ی کوتاه‌تر هستند به اشتباه به‌عنوان الگوهای متفاوت در نظر گرفته می‌شوند. این کار با تابعی مشابه محاسبه‌ی تابع شکست^{۱۴} در الگوریتم KMP انجام می‌شود (الگوریتم ۱.۴): برای رشته‌ی s ، طولانی‌ترین پیشوند سره که پسوند نیز هست محاسبه می‌شود؛ اگر طول رشته بر تفاضل طول رشته و این مقدار بخش‌پذیر باشد، آن تفاضل، طول کوچک‌ترین دوره‌ی تناوب رشته است.

¹¹ unsynch acts

¹² output trace

¹³ pattern

¹⁴ failure function

الگوریتم ۱۰.۴: `getMinimalPattern`: کاهش یک الگو به کوچک‌ترین دوره‌ی تناوب

ورودی: رشته‌ی خروجی `pattern`

خروجی: کوچک‌ترین دوره‌ی تناوبِ معادلِ `pattern`

۱: $next[0] \leftarrow 0$

۲: برای $i = 1$ تا $|pattern| - 1$ انجام بده

۳: $next[i]$ را طبق تابع پیشوندی KMP روی `pattern` محاسبه کن

۴: پایان حلقه‌ی برای

۵: $smallPieceLen \leftarrow |pattern| - next[|pattern| - 1]$

۶: اگر $|pattern| \bmod smallPieceLen \neq 0$ آنگاه

۷: بازگردان `pattern` {الگو ذاتاً دوره‌ی کوتاه‌تری ندارد}

۸: پایان شرط اگر

۹: بازگردان `pattern[0 : smallPieceLen]`

۳.۴ ساخت اسکلت سازگار با الگو

با داشتن الگوی مینیمال (`BEFORE_LOOP`, `LOOP`)، تابع `generateRelatively` یک «اسکلت» اولیه از گراف مؤلفه می‌سازد (الگوریتم ۲.۴). ایده این است که برای هر نماد از پیشوند، یک یال جدید به سمت یک حالت تازه و کاملاً تصادفی (`chooseStateRandomly`)، از میان حالت‌های هنوز استفاده نشده کشیده شود، تا مسیر به یک حالت به نام `loopStartState` برسد؛ سپس همین کار برای نمادهای دوره‌ی تناوب تکرار می‌شود، با این تفاوت که مسیر در نهایت باید دوباره به `loopStartState` بازگردد و یک چرخه ببندد. از آنجا که در هر گام یک حالت کاملاً تصادفی جدید انتخاب می‌شود، شماره گذاری داخلی حالت‌ها در هر مؤلفه، مستقل و غیرقابل پیش‌بینی است؛ تنها ردِ خروجی حاصل از طی آن‌ها ثابت می‌ماند. در پایان، الگوی واقعاً ساخته شده دوباره با `generateSynchOutPattern` استخراج و با الگوی درخواستی مقایسه می‌شود؛ این ادعا^{۱۵} به عنوان یک آزمون صحت درون‌ساختی^{۱۶} عمل می‌کند.

الگوریتم ۲.۴ ساخت اسکلت گراف بر مبنای الگوی خروجی هم‌گام‌ساز

ورودی: الگوی (`BEFORE_LOOP`, `LOOP`)، تعداد کل حالت‌ها

خروجی: گرافی که ردِ خروجی آن از حالت ۰ دقیقاً برابر الگوست

۱: $freeStates \leftarrow \{1, \dots, numOfStates - 1\}$

۲: اگر $BEFORE_LOOP \neq \varepsilon$ آنگاه

۳: $loopStart \leftarrow$ یک حالت تصادفی از `freeStates` (و حذف آن از مجموعه)

۴: مسیری از حالت ۰ تا `loopStart` بساز که هر یال، یک نماد از `BEFORE_LOOP` را (به ترتیب) تولید کند و مقصد هر یال، حالتی تازه و تصادفی از `freeStates` باشد

۵: وگرنه

۶: $loopStart \leftarrow 0$

۷: پایان شرط اگر

۸: به همین ترتیب، چرخه‌ای از `loopStart` بساز که نمادهای `LOOP` را تولید کند و دوباره به `loopStart` برگردد

۹: {بررسی صحت:}

۱۰: `assert generateSynchOutPattern() = (BEFORE_LOOP, LOOP)`

¹⁵assertion

¹⁶built-in correctness oracle

۴.۴ الحاق حالت‌های اضافی بدون نقض الگو

هوشمندی اصلی طراحی، دقیقاً در همین بخش نهفته است. اگر تعداد حالت‌های درخواستی برای مؤلفه بیشتر از طول الگو (پیشوند + دوره) باشد، حالت‌های باقی‌مانده («آزاد») باید طوری به گراف متصل شوند که:

۱. ماشین روی کنش هم‌گام‌ساز کامل بماند (هر حالت، یک یال خروجی برای آن کنش داشته باشد)،
۲. اما ردِ خروجی دیده‌شده از حالت آغازین • همچنان دقیقاً همان الگوی ثابت باشد؛ حتی اگر این حالت‌های اضافی هرگز روی مسیر اصلی $loopStart \rightarrow loopStart$ قرار نگیرند.

راه‌حل (تابع‌های `generateExtras` و `generateExtraOuts`) این است که هر گروه از حالت‌های آزاد، به‌صورت یک مسیرِ خصوصیِ تصادفی به هم متصل می‌شوند تا سرانجام به یکی از حالت‌های «ثابت‌شده» (یعنی حالت‌هایی که از قبل در پیشوند یا در چرخه جای گرفته‌اند) بازگردند؛ سپس خروجیِ یال‌های همین مسیر خصوصی، پس‌رو^{۱۷} و بر اساس فاصله‌شان تا نقطه‌ی اتصال، از روی الگوی اصلی «بازپخش» می‌شود. الگوریتم ۳.۴ این رویه را خلاصه می‌کند.

الگوریتم ۳.۴ الحاق حالت‌های اضافی به اسکلت سازگار با الگو

- ورودی:** اسکلت ساخته‌شده در الگوریتم ۲.۴، مجموعه‌ی `freeStates` باقی‌مانده
- خروجی:** گرافی کامل روی کنش هم‌گام‌ساز که هم‌چنان الگوی خروجیِ آن از حالت • حفظ شده باشد
۱. تا زمانی که $freeStates \neq \emptyset$ انجام بده
 ۲. $cur \leftarrow$ یک حالت تصادفی از `freeStates`؛ آن را از مجموعه حذف کن
 ۳. $states \leftarrow [cur]$
 ۴. $choosable \leftarrow$ حالت‌های «ثابت‌شده» (دارای موقعیت مشخص در پیشوند/چرخه)
 ۵. **تکرار کن**
 ۶. $next \leftarrow$ انتخاب تصادفی از `choosable`
 ۷. یال کنش هم‌گام‌ساز را از `cur` به `next` بکش (خروجی هنوز نامشخص)
 ۸. اگر $next \in freeStates$ آنگاه
 ۹. `next` را از `freeStates` حذف کن و به `states` اضافه کن
 ۱۰. `choosable` را به‌روزرسانی کن: علاوه بر حالت‌های آزاد، فقط حالت‌های `states[i]` که $i \bmod |LOOP| = 0$ است را نیز اضافه کن (این قید تضمین می‌کند اگر مسیر خصوصی به چرخه‌ای جدید تبدیل شود، طول آن چرخه مضربی از دوره‌ی تناوب اصلی باشد و بتواند برچسب‌گذاری سازگار بپذیرد)
 ۱۱. **وگرنه**
 ۱۲. `generateExtraOuts(states)` {بازپخش الگو به‌صورت پس‌رو، شرح در متن}
 ۱۳. پایان این شاخه
 ۱۴. **پایان شرط اگر**
 ۱۵. $cur \leftarrow next$
 ۱۶. تا زمانی که شاخه به یک حالت ثابت‌شده برسد
 ۱۷. **پایان حلقه‌ی تا زمانی که**

تابع `generateExtraOuts` برای شاخه‌ای که به یک حالت ثابت (نقطه‌ی اتصال) رسیده است، ابتدا مشخص می‌کند این نقطه‌ی اتصال، معادل کدام «فاز» از الگوی سراسری است (با شمارش تعداد گام از حالت • تا آن نقطه، یا در صورتی که شاخه خودش یک چرخه‌ی تازه تشکیل داده، با تشخیص نقطه‌ی شروع همان چرخه‌ی جدید). سپس، با حرکت پس‌رو از نقطه‌ی اتصال به سمت آغاز شاخه، به هر یال، خروجیِ همان فاز از الگوی اصلی را نسبت می‌دهد. به این ترتیب،

¹⁷backwards

مسیر خصوصی از دید یک ناظر بیرونی (مؤلفه‌ی هم‌گام‌ساز دیگر) هرگز قابل تشخیص از مسیر اصلی نیست: هر بار که این مؤلفه به فاز مشخصی از الگو می‌رسد — چه از مسیر اصلی، چه از میان یکی از شاخه‌های خصوصی فراوانش — همان خروجی ثابت را تولید می‌کند. پس از این مرحله، بررسی‌های استاندارد صحت ماشین حالت (که پیش از این نیز در تولیدکننده‌ی SCL-Star وجود داشت) اجرا می‌شوند: قابلیت دسترسی همه‌ی حالت‌ها (isEveryStateReachable)، کمینه‌بودن گراف (isGraphMinimal) و اثرگذاری واقعی همه‌ی کنش‌ها (isEveryActEffective)؛ در صورت نقض هرکدام، کل فرایند تکرار می‌شود.

۵.۴ خنثی‌سازی اثر کنش‌های خصوصی بر الگوی هم‌گامی

پس از ساخت اسکلت روی کنش هم‌گام‌ساز، کنش‌های خصوصی هر مؤلفه (unsynchActs) نیز باید اضافه شوند. برای آنکه این کنش‌ها به هیچ وجه رفتار کنش هم‌گام‌ساز را تغییر ندهند، مقصد هر یال خصوصی تنها از میان حالت‌هایی انتخاب می‌شود که با حالت مبدأ، از نظر ردِ آتی کنش هم‌گام‌ساز هم‌ارز^{۱۸} هستند (مجموعه‌ی equalStates، که با تابع بازگشتی areFunctionallySameStates محاسبه می‌شود). به این ترتیب، افزودن کنش‌های خصوصی هرگز الگوی خروجی سراسری کنش هم‌گام‌ساز را نقض نمی‌کند.

۶.۴ تصادفی‌سازی نوع مؤلفه‌ها در آزمون‌های نقطه‌به‌نقطه

در تولیدکننده‌ی سناریوهای نقطه‌به‌نقطه^{۱۹}، برای پوشش هم‌زمان دو حالت خروجی ایستا (مطابق SCL-Star پیشین) و خروجی پویا (جدید)، تعدادی از جفت مؤلفه‌ها به‌طور تصادفی با خروجی پویا (staticSynchOut=False) و باقی با خروجی ایستا تولید می‌شوند. این کار تضمین می‌کند مجموعه‌ی آزمون‌ها هم به لحاظ رگرسیون (حفظ درستی رفتار پیشین) و هم به لحاظ ویژگی جدید، جامع باشد.

۵ آزمایش‌ها و نتایج بر روی آزمون‌های تولیدشده

پس از تعمیم تولیدکننده، آزمون‌های تازه برای چهار توپولوژی Point-to-Point، Mesh، Ring و Star تولید و الگوریتم ESCL-Star روی آن‌ها اجرا شد. برای هر توپولوژی، عملکرد یادگیری ترکیبی (CLSTAR) با یادگیری یکپارچه‌ی مستقیم با L* (LSTAR) بر حسب تعداد نمادهای پرسش عضویت^{۲۰} و تعداد بازنشانی‌ها^{۲۱} مقایسه شد. جدول ۱ میانگین این معیارها را برای آزمون‌های پیشین (خروجی ایستا) در برابر آزمون‌های تازه (خروجی پویا) نشان می‌دهد.

¹⁸functionally equivalent

¹⁹Point-to-Point

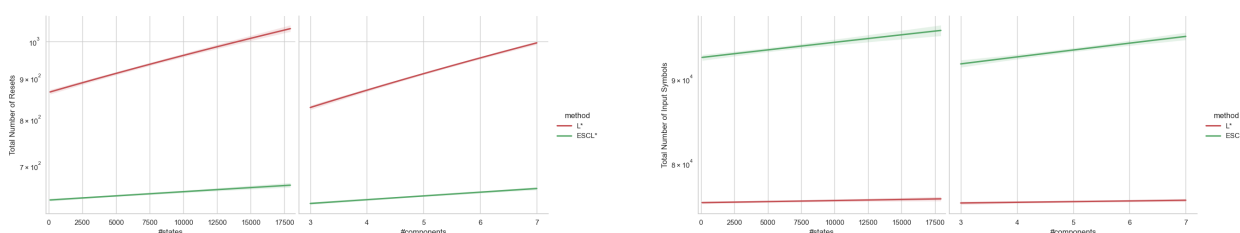
²⁰membership query symbols

²¹resets

جدول ۱: میانگین معیارهای یادگیری روی آزمون‌های تولیدشده: مقایسه‌ی آزمون‌های پیشین (خروجی ایستا) با آزمون‌های تازه (خروجی پویا)

تعداد آزمون	نسبت بهبود	CLSTAR (نماد)	LSTAR (نماد)	نوع آزمون	توپولوژی
255	$\times 13/7$	3,174,089	43,333,265	ایستا (پیشین)	Point-to-Point
222	$\times 5/7$	5,005,844	28,306,775	پویا (تازه)	Point-to-Point
147	$\times 3/1$	22,769,066	71,313,431	ایستا (پیشین)	Mesh
147	$\times 5/4$	111,997,639	600,415,832	پویا (تازه)	Mesh
240	$\times 11/1$	8,867,158	98,289,392	ایستا (پیشین)	Ring
209	$\times 7/0$	16,217,499	114,245,936	پویا (تازه)	Ring
213	$\times 6/2$	6,749,016	42,031,831	ایستا (پیشین)	Star
144	$\times 3/9$	12,611,225	49,531,085	پویا (تازه)	Star

همان‌طور که جدول ۱ نشان می‌دهد، اجازه دادن به خروجی پویا برای کنش‌های هم‌گام‌ساز، هزینه‌ی مطلق یادگیری (چه برای LSTAR و چه برای CLSTAR) را افزایش می‌دهد؛ زیرا الگوریتم دیگر نمی‌تواند صرفاً با مشاهده‌ی یک خروجی متفاوت، کنش را فوراً ناهم‌گام تشخیص دهد و باید مجموعه‌های الفبایی را به‌طور جزئی یاد بگیرد و در ProductMealy بررسی کند. با این‌همه، در تمام توپولوژی‌ها، برتری یادگیری ترکیبی نسبت به یادگیری یکپارچه (نسبت بهبود بین ۳/۹ تا ۷ برابر برای آزمون‌های تازه) هم‌چنان پابرجاست؛ یعنی توسعه‌ی الگوریتم، مزیت ذاتی رویکرد ترکیبی را از بین نبرده است. نمودارهای شکل ۳ روند مقایسه‌ای تعداد نمادها و بازنشانی‌ها را در سراسر آزمون‌های ادغام‌شده‌ی چهار توپولوژی نشان می‌دهند.



شکل ۳: مقایسه‌ی تعداد نمادهای پرسش عضویت (چپ) و تعداد بازنشانی‌ها (راست) برای LSTAR در برابر ESCL-Star روی مجموعه‌ی ادغام‌شده‌ی آزمون‌های تولیدشده.

۶ بنچمارک واقعی: bankCard

در گام پایانی، به‌جای تکیه‌ی صرف بر آزمون‌های تولیدشده، مجموعه‌ای از مدل‌های واقعی آموخته‌شده از پروتکل کارت‌های بانکی^{۲۲} به‌عنوان بنچمارک جدید انتخاب شد. این مجموعه شامل ۱۱ مؤلفه است که هرکدام رفتار یک نوع کارت یا برنامه‌ی کاربردی بانکی (از جمله MasterCard، MAESTRO، PIN، SecureCode، و نمونه‌های ASN، Rabo و Volksbank) را نمایش می‌دهند و جایگزین بنچمارک قبلی (مبتنی بر پروتکل BCS) شدند. انتخاب bankCard به این دلیل بود که کنش‌های اشتراکی چند مؤلفه در این پروتکل واقعی، نمونه‌ی طبیعی‌ای از کنش‌های هم‌گام‌ساز با خروجی وابسته به وضعیت هستند و بستری مناسب برای سنجش ESCL-Star در شرایط واقعی فراهم می‌کنند.

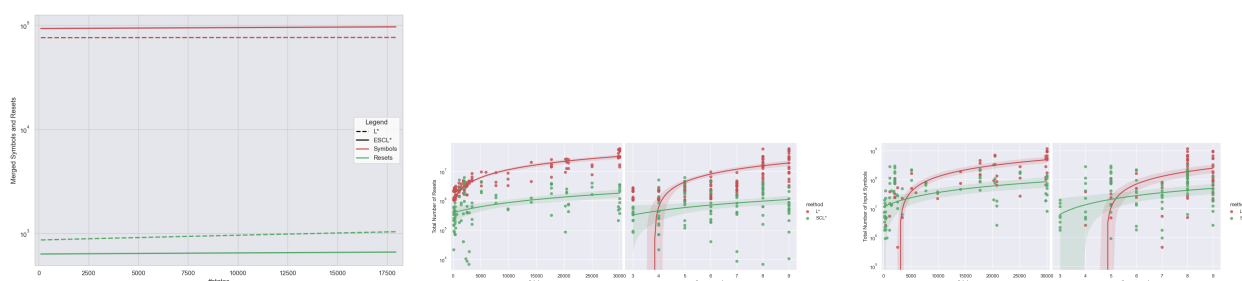
علاوه بر این، برای افزایش جامعیت بنچمارک، به هر یک از مؤلفه‌های bankCard یک کنش خصوصی (ناهم‌گام) اضافه شد تا هر مؤلفه، علاوه بر کنش‌های پروتکلی مشترک، حداقل یک رفتار محلی و غیرقابل مشاهده از بیرون نیز

²²bankCard

داشته باشد؛ این کار پوشش بخش «خنثی سازی اثر کنش های خصوصی» (بخش ۴) را در یک محیط واقعی نیز می آزماید. همچنین منطق انتخاب زیرمجموعه ی مؤلفه ها برای هر آزمون (ChooseTests.py) ساده سازی شد تا به جای گروه بندی از پیش تعیین شده ی مؤلفه های هم گام (که در بنچمارک BCS استفاده می شد)، هر زیرمجموعه ای از مؤلفه های bankCard بتواند در کنار هم آزموده شود. جدول ۲ خلاصه ی نتایج اجرای ESCL-Star روی این بنچمارک را در برابر بنچمارک واقعی پیشین نشان می دهد و شکل ۴ نمودارهای مقایسه ای متناظر را نمایش می دهد.

جدول ۲: میانگین معیارهای یادگیری روی بنچمارک واقعی: بنچمارک پیشین در برابر bankCard

بنچمارک واقعی	LSTAR (نماد)	CLSTAR (نماد)	نسبت بهبود	تعداد آزمون
بنچمارک پیشین (BCS) bankCard (تازه، با کنش خصوصی افزوده)	39,392,529	20,321,058	$\times 1/9$	227
	70,189,184	28,054,821	$\times 2/5$	147



شکل ۴: نمودارهای مقایسه ی تعداد نمادها، تعداد بازنشانی ها، و نمای ادغام شده برای ESCL-Star روی بنچمارک واقعی bankCard.

همان گونه که در جدول ۲ دیده می شود، بنچمارک bankCard به دلیل بزرگ تر بودن مؤلفه ها (تعداد حالت ها معمولاً بین ۱۴۰ تا ۲۲۴) و افزودن کنش خصوصی به هر مؤلفه، هزینه ی مطلق یادگیری را نسبت به بنچمارک پیشین افزایش داده است؛ اما نسبت بهبود یادگیری ترکیبی نسبت به یادگیری یکپارچه ($\times 2/5$) نه تنها حفظ، بلکه نسبت به بنچمارک قبلی ($\times 1/9$) بهبود یافته است. این نتیجه نشان می دهد در سامانه های واقعی بزرگ تر و پیچیده تر، مزیت رویکرد ترکیبی ESCL-Star پررنگ تر می شود.

۷ جمع بندی

در این گزارش، توسعه ی الگوریتم SCL-Star به ESCL-Star برای پشتیبانی از کنش های هم گام ساز با خروجی پویا (اما سازگار در نقاط هم گامی) شرح داده شد. تغییر اصلی سمت یادگیری، جایگزینی قاعده ی «حذف فوری کنش از sync به محض مشاهده ی خروجی متفاوت» با رویه ای مبتنی بر یادگیری جزئی و بررسی صحت هم گامی در ProductMealy بود. سمت دوم و دشوارتر کار، تعمیم تولیدکننده ی آزمون های خودکار بود تا بتواند مؤلفه های تصادفی متنوعی تولید کند که رد خروجی کنش هم گام سازشان، صرف نظر از تفاوت توپولوژی و اندازه، دقیقاً با یک الگوی مشترک (پیشوند + دوره ی تناوب) منطبق بماند؛ حتی هنگام افزودن حالت ها و کنش های خصوصی اضافی. نتایج آزمایش ها، هم روی آزمون های تولید شده و هم روی بنچمارک واقعی bankCard، نشان داد که مزیت یادگیری ترکیبی نسبت به یادگیری یکپارچه، پس از این تعمیم همچنان حفظ می شود.